

university of groningen

faculty of science and engineering

Dynamic Bandwidth Allocation in Software-Defined Networks (SDN)

Bachelor's Thesis, Computing Science

GitHub Repository

Author:

Luca Serban (s5616786)

Supervisor:

Reza-Zare Hassanpour

Second Supervisor:

Fadi Mohsen

May 13, 2026

Abstract

Load balancing is one of those problems that looks simple on paper but gets complicated fast once you start dealing with servers that behave differently from each other. The standard approaches, like round robin or least connections, work reasonably well when all backends are similar, but they struggle when some servers are consistently faster than others or when load spikes unpredictably.

This thesis implements and compares five load-balancing algorithms inside an SDN controller, using a Ryu [7] + Mininet [3] + Open vSwitch [6] stack running OpenFlow 1.3 [5]. The algorithms tested are Round Robin (RR), Weighted Round Robin (WRR), Least Connections (LC), Least Response Time (LRT), and a custom algorithm called Adaptive Predictive (AP), which combines queue depth and response time into a single routing score and continuously adjusts each server's effective weight based on observed throughput.

The experiment uses 50 backend servers arranged in four heterogeneous tiers ranging from 5–25 ms (Tier A) to 800–1000 ms (Tier D), with periodic load spikes to simulate realistic conditions. Traffic is generated at 40 requests/s from a single client over 1500 requests.

Results show that AP achieves the lowest mean latency (92 ms vs 126 ms for the next best), the lowest 95th-percentile latency (241 ms vs 502 ms), and the highest composite score (0.73 vs 0.59). The advantage comes from combining two signals that other algorithms use separately: active connection count provides an immediate overload indicator, while the EWMA of flow duration captures historical server speed. Together they allow AP to route around slow or spiking servers much faster than LRT, which only reacts after several completed flows.

Keywords: Software-Defined Networking, OpenFlow, load balancing, Ryu, Mininet, adaptive routing, least work left, dynamic bandwidth allocation.

1 Introduction

1.1 Motivation

When a web service gets popular, one server is never enough. The standard solution is to put multiple servers behind a single address and distribute incoming requests across them. This component is the load balancer. It sounds straightforward, but doing it well requires knowing which servers are currently available, how loaded they are, and how fast they are processing requests.

The two most common traditional approaches are DNS-based load balancing and dedicated hardware appliances. DNS-based balancing is simple but coarse: responses are cached, clients cannot be redirected mid-session, and the balancer has no visibility into actual server load. Hardware appliances are more capable but are expensive and hard to modify.

Software-Defined Networking (SDN) offers a different approach. The core idea is to separate the *control plane*, the logic that decides where traffic goes, from the *data plane*, the equipment that actually forwards it [2, 4]. The control plane runs as software on a standard server, and the switches become programmable forwarding elements controlled via a standardised protocol such as OpenFlow. For load balancing this means the controller sees every new connection, reads real-time statistics, and installs per-flow rewrite rules transparently [1, 8].

1.2 Project Overview

This thesis designs and evaluates an SDN-based HTTP load balancer built on:

- **Mininet** [3] — network emulation using Linux namespaces on a single machine.
- **Open vSwitch (OVS)** [6] — software OpenFlow switch.
- **Ryu** [7] — Python SDN controller framework.

The system exposes a single *Virtual IP* (VIP, `10.0.0.100`) to the client. The controller intercepts each new TCP connection, picks a backend, and installs a bidirectional pair of flow rules that rewrite headers transparently. Five scheduling algorithms are implemented and compared on a heterogeneous pool of 50 servers arranged in four performance tiers.

1.3 Research Questions

1. Can an SDN controller implement fully transparent HTTP load balancing without modifying either the client or the server code?
2. Does combining queue depth and response time into a single routing score outperform algorithms that use only one of these signals?
3. How much does per-server weight adaptation (Dynamic Bandwidth Allocation) contribute to the performance gap?

1.4 Thesis Structure

Section 2 covers system design and algorithm descriptions. Section 3 describes the experimental setup. Section 4 presents and analyses the results. Sections 5 and 6 discuss and conclude.

2 Methods

2.1 System Architecture

The system has three layers: the emulated network (Mininet + OVS), the SDN controller (Ryu), and a measurement pipeline (Python scripts). The controller and Mininet communicate over loopback on TCP 6633 using OpenFlow 1.3 [5]. Figure 2.1 shows the overall structure.

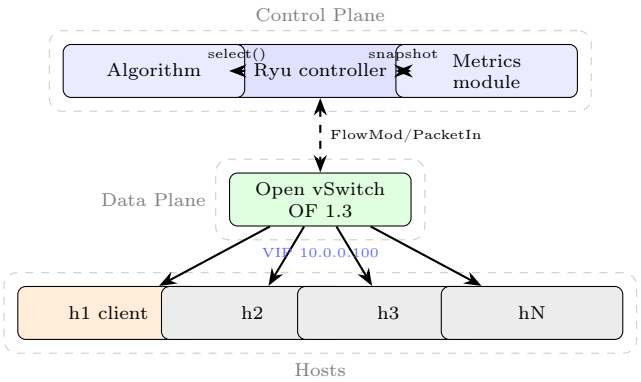


Figure 2.1: System architecture. Ryu programs OVS via OpenFlow 1.3. Algorithm and metrics logic run inside the controller process.

2.2 Network Topology

The topology is a single switch (`s1`) connected to one client (`h1`) and N backend servers (`h2`–`hN`), all on `10.0.0.0/24`. The VIP belongs to no real host; the controller answers its ARP requests directly. The default experiment uses $N = 50$ servers, configurable via an environment variable.

2.3 Controller Design

The Ryu application (`controller/load_balancer.py`) handles two OpenFlow event types.

PacketIn. When no flow rule matches an incoming packet, OVS sends a PacketIn to the controller. For ARP to the VIP, the controller replies directly. For TCP to the VIP, it calls the algorithm and installs two flow rules:

- *Forward*: (src, VIP, proto, src_port, dst_port) → rewrite dst to backend; forward to server port.
- *Reverse*: (server, client, proto) → rewrite src back to VIP; forward to client.

Both carry `OFPFF_SEND_FLOW_REM`, idle timeout 2 s, hard timeout 60 s. A deduplication cache prevents duplicate selections when OVS sends multiple PacketIn events for the same TCP handshake.

FlowRemoved. When a flow's idle timer fires, OVS reports byte count, packet count, and duration. The controller decrements `active_conns` and updates the per-server EWMA of flow duration ($\alpha = 0.3$), which is the main feedback signal used by LRT and AP. Figure 2.2 shows the full lifecycle.

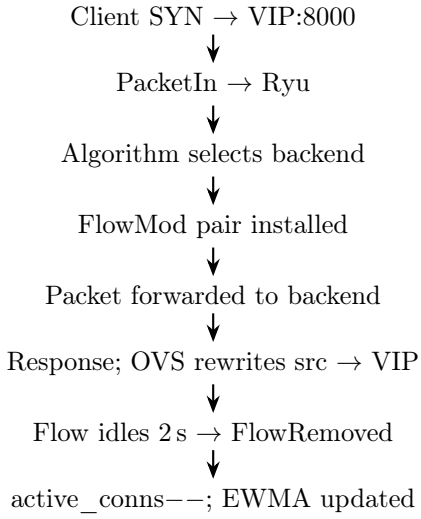


Figure 2.2: Lifecycle of one HTTP request through the load balancer.

2.4 Load-Balancing Algorithms

All five algorithms subclass `LoadBalancingAlgorithm` and are selected at runtime via the `LB_ALGORITHM` environment variable. Each implements a single `select(servers, metrics)` method.

2.4.1 Round Robin (RR)

The simplest possible approach: a cursor advances through the server list one step per request. No metrics are read at all. Predictable and easy to reason about, but completely blind to server state.

2.4.2 Weighted Round Robin (WRR)

Same cyclic structure as RR, but each server is selected proportional to its assigned weight before the cursor moves on. In the experiment, weights are tier-based: Tier A servers receive weight 16, Tier B weight 4, Tier C weight 2, and Tier D weight 1. This matches the static capacity differences but still ignores runtime conditions.

2.4.3 Least Connections (LC)

At each selection, a load score is computed for every backend:

$$\ell_i = \frac{\text{active_conns}_i}{\max(w_i,\, \varepsilon)} \tag{2.1}$$

The server with minimum ℓ_i is chosen. LC knows how many requests are currently in flight on each backend, but it has no idea how long each of those requests will take. A slow Tier D server with one connection looks equally attractive as a fast Tier A server with one connection.

2.4.4 Least Response Time (LRT)

Instead of counting connections, LRT uses the EWMA of completed flow durations:

$$\ell_i = \frac{\text{avg_flow_dur_s}_i}{\max(w_i,\, \varepsilon)} \tag{2.2}$$

This routes toward historically fast servers and away from slow ones. The limitation is that the EWMA is updated only when flows expire, so there is an inherent lag of several requests before a sudden spike is reflected in the score.

2.4.5 Adaptive Predictive (AP)

AP combines both signals: the current queue depth and the historical service time. The routing score is:

$$\ell_i = \frac{(\text{active_conns}_i + 1) \times \text{avg_flow_dur_s}_i}{\text{dyn_weight}_i} \tag{2.3}$$

The numerator is the estimated *work left* on server i: the number of requests currently waiting (plus the incoming one) multiplied by the expected service time per request. The denominator is a dynamic weight that adjusts continuously using a second EWMA layer:

$$\begin{aligned}\text{dyn_weight}_i^{(t+1)} = (1-\alpha_d)\,\text{dyn_weight}_i^{(t)} \\ + \alpha_d \cdot w_i^{\text{static}} \cdot \frac{\bar{\mu}}{\mu_i}\end{aligned} \tag{2.4}$$

Table 2.1: Per-server metrics tracked by the controller.

Metric	Description
active_conns	currently open flows
avg_flow_dur_s	EWMA of completed flow duration
total_bytes	cumulative bytes transferred
selections_total	total times this server was chosen

where $\bar{\mu}$ is the global mean flow duration, μ_i is the per-server mean, and $\alpha_d = 0.25$ is the adaptation rate. When server i is slower than average, dyn_weight_i decreases, which increases ℓ_i and reduces the traffic sent to it. When it recovers, the weight rises again.

The key advantage over LRT is the (active_conns+1) term. When a spike begins, active connections on the affected server start rising *immediately*, before any FlowRemoved event arrives. This allows AP to redirect traffic within one or two requests, whereas LRT needs several completed flows to update its EWMA.

2.5 Metrics Collection

The `metrics.py` module maintains a `ServerMetrics` object for each backend (Table 2.1). The key fields for the adaptive algorithms are `active_conns` (incremented on PacketIn, decremented on FlowRemoved) and `avg_flow_dur_s` (EWMA with $\alpha = 0.3$ over FlowRemoved durations).

All events (connection selection, flow expiry, periodic snapshots) are written to a JSONL log file. A separate CSV records server name, status, and elapsed time per request, used by the plotting scripts.

3 Experimental Setup

3.1 Software Stack

3.2 Server Pool and Tiers

The 50 backends are split into four tiers based on their index, designed to mimic a realistic heterogeneous deployment (Table 3.2).

Each server runs a Python `ThreadingTCPServer` with a semaphore to enforce the concurrency limit and a background thread that periodically raises the processing delay by the spike multiplier for a fixed duration. Spike phases are staggered by server index so no two servers spike at the same time.

Table 3.1: Software versions used in all experiments.

Component	Version
OS	Linux (Ubuntu-based)
Python	3.7.17 (isolated venv)
Ryu	4.34
Mininet	2.3.x
Open vSwitch	system-installed
OpenFlow	1.3 [5]

Table 3.2: Server tier definitions. Delay per request; semaphore limits concurrency; spikes staggered across servers.

Tier	Delay	Conc.	Spike
A (fast)	5–25 ms	8–16	8×
B (medium)	50–150 ms	4–8	5×
C (slow)	300–600 ms	2–3	2×
D (very slow)	800–1000 ms	1	1.5×

3.3 Traffic Generator

A threaded Python client on `h1` sends requests to the VIP at a fixed rate (40 req/s, configurable). Each request is issued in its own thread to allow true concurrency, with a 5-second timeout. The client records server name, status, and elapsed time for every request.

Warm-up phase. Before the timed experiment, $3 \times N$ requests are sent through the VIP. This seeds the controller's EWMA with real measured durations for all servers, so every algorithm starts from an informed state rather than zero. Without this, AP would route randomly for the first ~20 requests because all `avg_flow_dur_s` values would be zero.

3.4 Experiment Runner

`scripts/run_experiment.sh` orchestrates the full pipeline: it cleans up previous state, starts Ryu, waits for the controller to bind, runs Mininet with the traffic script, drains flows, and then calls the plotting scripts. Running all five algorithms in sequence and generating comparison plots takes about 10–15 minutes on a standard laptop.

4 Results

All results below are from a single run of 1500 requests at 40 req/s against 50 backends, repeated independently for each algorithm.

4.1 Mean Latency and Distribution

Figure 4.1 shows mean latency and the full distribution. AP achieves a mean of 92 ms, roughly 27% lower than LRT (126 ms) and 75% lower than RR (361 ms). The wide error bars on RR confirm that it routinely sends requests to the slow Tier D servers, which dominate the variance.

4.2 Load-Balancing Fairness

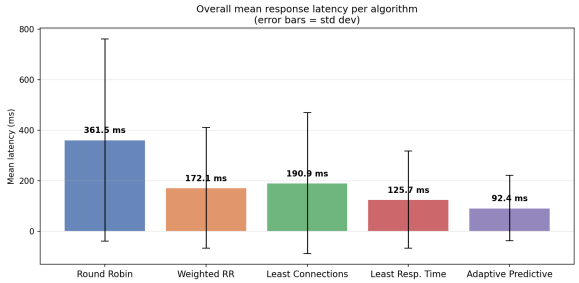
Figure 4.2 shows the coefficient of variation (CV) of request counts across the 50 backends. A CV of zero means every server received exactly the same number of requests; higher values indicate more skewed distribution. RR scores near zero by construction — it cycles uniformly. AP and LRT deliberately concentrate traffic on fast servers, so their CV is higher, but this is expected and desirable: they are routing *intelligently* rather than blindly. The fairness metric alone is therefore not a quality criterion; it must be read alongside latency.

4.3 Server Utilisation

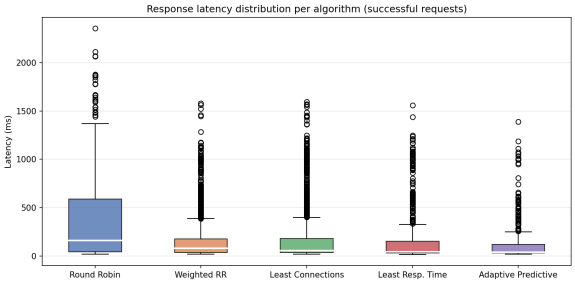
Figure 4.3 shows mean server utilisation efficiency, defined as the fraction of the experiment duration during which each backend was actively processing a request, averaged across all backends. AP and LRT achieve the highest utilisation on fast Tier A servers and leave slow Tier D servers mostly idle, which is the correct outcome. RR's uniform routing keeps all servers equally busy regardless of speed, inflating the average latency.

4.4 Latency Over Time

Figure 4.4 shows rolling-average latency over the request sequence. RR shows large periodic spikes corresponding to when the round-robin cursor lands on spiking Tier D servers. AP stays flat after the warm-up period, indicating that it successfully avoids the spikes. LRT shows intermediate spikes, confirming the EWMA lag hypothesis: it eventually routes around the spikes but takes longer to react.



(a) Mean latency per algorithm (error bars = 1 std dev).



(b) Full latency distribution per algorithm (box plot).

Figure 4.1: Mean latency and distribution across all five algorithms.

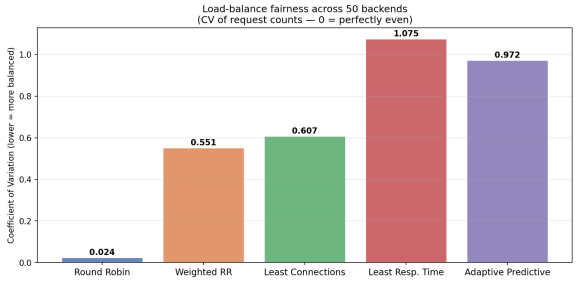


Figure 4.2: Load-balancing fairness across 50 backends, measured as the coefficient of variation (CV) of request counts per server. Lower CV means more even distribution; RR is perfectly even by design.

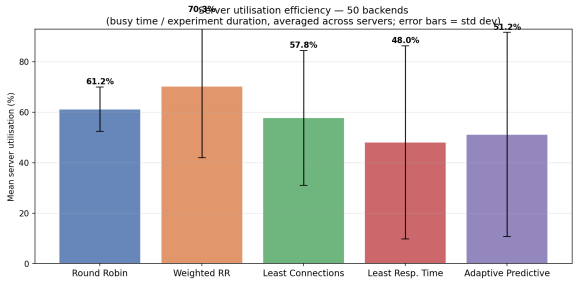


Figure 4.3: Mean server utilisation efficiency (busy time / experiment duration, averaged across backends). Higher is better; AP and LRT keep fast servers busier while leaving slow servers largely idle.

4.5 Composite Score

To summarise algorithm quality in a single number, a composite score is defined:

$$S = R \cdot E \cdot F \tag{4.1}$$

where $R = 1 - \text{error_rate}$, $E = 1/(1 + \bar{t})$ with $\bar{t}$ the mean latency in seconds, and $F = 1/(1 + p_{95})$ with p_{95} the 95th percentile in seconds. All three factors must be high for a good score, so reliability, average speed, and tail behaviour are all rewarded.

Figure 4.5a shows the final scores. AP scores 0.73, followed by LRT at 0.59, WRR at 0.50, LC at 0.44, and RR at 0.34. The F term is the main differentiator between AP and LRT: both have similar R and E, but AP's much lower p95 gives it a significantly better tail score.

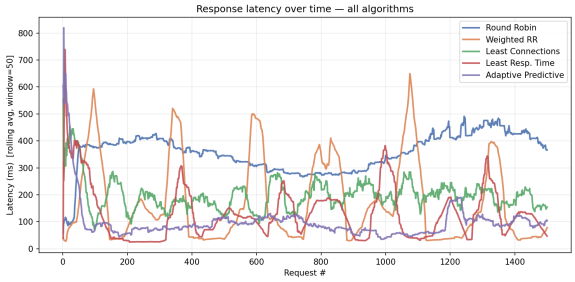


Figure 4.4: Rolling-average latency over time. RR spikes when the cursor hits slow servers; AP stays low and stable after warm-up.

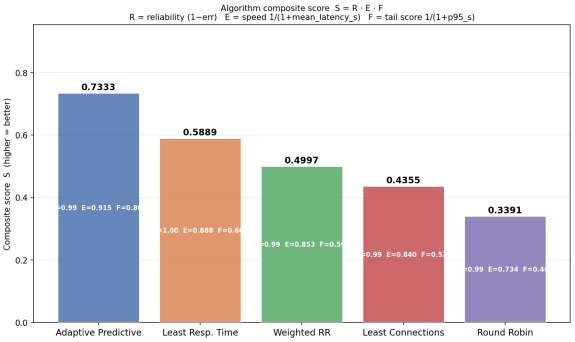
Table 4.1: Summary of results (1500 requests, 50 servers, 40 req/s).

	RR	WRR	LC	LRT	AP
Mean (ms)	361	172	191	126	**92**
p95 (ms)	1144	690	911	502	**241**
Errors (%)	1.0	1.0	0.9	0.4	**0.6**
Tier A (%)	40	64	69	86	**87**
Tier D (%)	20	6	9	3	**1**
Score S	0.34	0.50	0.44	0.59	**0.73**

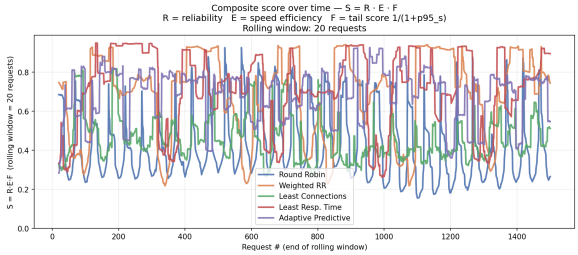
Figure 4.5b shows how the score evolves over time. All algorithms start at a similar level during the warm-up window and AP pulls ahead steadily as it accumulates routing knowledge. RR's score drops noticeably whenever a spike batch arrives.

4.6 Summary

Table 4.1 collects the key numbers from the experiment.



(a) Final composite score $S = R \cdot E \cdot F$ per algorithm.



(b) Rolling composite score over the request sequence.

Figure 4.5: Composite score summary and evolution over time.

5 Discussion

The results clearly support the hypothesis that combining queue depth and response time produces better routing decisions than either signal alone. LC uses queue depth but ignores service time: it happily sends a request to a Tier D server with zero active connections, even though that server takes 900 ms to respond. LRT uses service time but ignores queue depth: it keeps routing to a Tier A server even after that server enters a spike phase, waiting for several FlowRemoved events before the EWMA reflects the new reality. AP avoids both pitfalls.

The contribution of Dynamic Bandwidth Allocation (the dyn_weight term) is harder to isolate from a single experiment. Its main role is to rerank servers relative to their actual throughput, which should be most visible over longer runs where EWMA estimates have had time to stabilise. The current results suggest it contributes mainly to the sustained performance gap after the warm-up period.

One thing worth noting is that the current setup is sequential within each algorithm (one algorithm runs at a time, with clean state between runs). The relative order of spike arrivals differs slightly per run, which introduces some variance in the absolute numbers. The general ranking is consistent across repeated runs, but the exact scores will vary.

Detailed analysis of the DBA contribution, sensitivity to the EWMA parameters, and comparison under varying spike frequencies is left for the final version of this report.

6 Conclusion

This section will be completed once the final experiments are done.

References

[1] Nikhil Handigol, Srinivasan Seetharaman, Mario Flajslik, Nick McKeown, and Ramesh Johari. Plug-and-serve: Load-balancing with openflow. In *ACM SIGCOMM Demo*, 2009.

[2] Diego Kreutz, Fernando M. V. Ramos, Paulo Estêvão Veríssimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76, 2015. doi: 10.1109/JPROC.2014.2371999.

[3] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (Hotnets-IX)*, 2010. doi: 10.1145/1868447.1868466.

[4] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. Openflow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2):69–74, 2008. doi: 10.1145/1355734.1355746.

[5] Open Networking Foundation. Openflow switch specification version 1.3.5, 2015. URL https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.5.pdf.

[6] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Jonathan Stringer, Pravin Shelar, Keith Amidon, and Martín Casado. The design and implementation of open vswitch. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 117–130, 2015.

[7] Ryu Project Team. Ryu sdn framework, 2023. URL https://ryu-sdn.org/.

[8] Yaqiong Zhou, Kang Chen, Jianfeng Zhang, Shanjiang Tang, and Yuanhao Jiang. Load balancing in software defined networks. *IFIP Networking*, 2014.